Space Robotics – Assignment 3

Lucas Plumbohm
24949202

Elijah Spannenberg
24767473

Mathis Beissner
26665176

1 Generative AI Disclosure

Generative AI tools were used to assist with analysing error messages during the debugging process, creating helper scripts for advanced visual debugging, formatting this report as a LaTeX document, and correcting spelling and wording throughout the report.

2 Statement of Individual Contributions

Task	Completed?	Team member	Contribution
Perception 1	Yes	Mathis	All
Perception 2	Yes	Mathis	All
Perception 3	Yes	Mathis	All
Planning 1	Yes	Lucas	All
Planning 2	Yes	Lucas	All
Planning 3	Yes	Lucas	All
Advanced 1			
Advanced 2			
Advanced 3			
Advanced 4			
Advanced 5			
Advanced 6			
Report writing			
Recorded videos			

3 Project Overview

TBA

4 Implementation

TBA

4.1 Perception 1: Collect an Image Dataset of the Artefacts

The goal of this task was to create an image dataset for artefact detection. A subscriber was added to the `CaveExplorer` node so that image capture could be requested by manually publishing a `std_msgs/msg/Empty` message. The callback below sets a flag, prompting the next image callback to save the images and reset the flag. A new depth subscriber was synchronised with the existing RGB subscriber so that each matching image pair is processed by a single callback.

```
    def capture_callback(self, msg):
        self.capture_requested_ = True
        self.get_logger().info('Requesting image capture.')
```

Listing 1: Implementation of capture callback

To investigate how many images were needed, test models were trained on subsets containing 10, 20 and 30 images of the alien artefact only. Performance improved with dataset size, with the 30-image dataset producing the best results among the tested sizes. Precision and recall both reached 1.0 on the test split (Figure 1). Based on this result, at least 30 images were collected per artefact class. The model and training procedure are described in more detail in Perception 2.

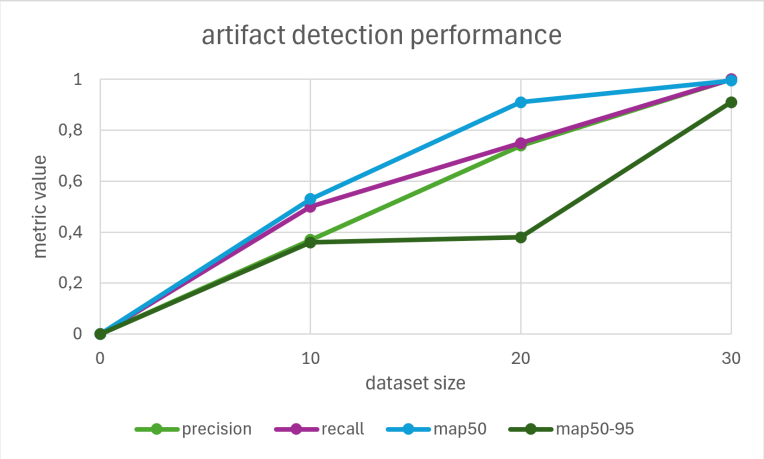


Figure 1: Detection performance for the alien artefact with increasing dataset size. Precision and recall reach 1.0 on the test split at 30 images.

The final dataset contains 203 images, including scenes with multiple artefacts and a few negative examples without artefacts to help reduce false positives. Some images contain multiple instances of a class, resulting in

more than 30 instances per class. Figure 2 shows a corresponding RGB and depth image pair containing the gem and mushroom artefacts.

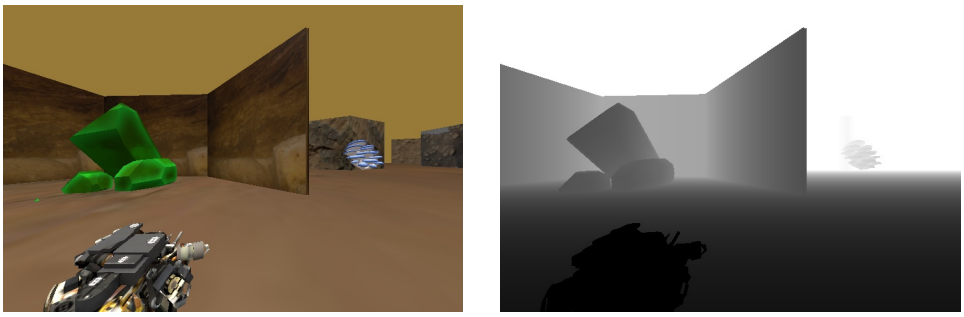


Figure 2: Example RGB image (left) and corresponding depth image (right) from the collected dataset, showing the gem and mushroom artefacts.

4.2 Perception 2: Detect Artefacts with Computer Vision

The goal of this task was to train and integrate an artefact detector using the dataset collected in Perception 1. The Ultralytics YOLO framework was used with a pretrained YOLO26 nano model to keep inference time and computational requirements low. The data was divided into training, validation and test sets for updating model weights, monitoring training and evaluating the final model, respectively. Several hyperparameter combinations were tested before selecting a learning rate of 0.002, a batch size of 4 and the AdamW optimiser.

Separate models were trained for RGB and depth images. For depth detection, distances from 0 to 15 metres were mapped to greyscale values to improve the visual contrast of the artefacts. Annotations beyond this range were removed. Stop-sign annotations also required removal because the sign is a texture on a flat surface and cannot be distinguished in depth images.

Table 1 summarises the saved test results for the final RGB and depth models. The depth model achieved a precision of 1.0 and a higher mAP@50–95, while the RGB model achieved slightly higher recall and mAP@50.

Metric	RGB model	Depth model
Precision	0.985	1.000
Recall	0.981	0.975
mAP@50	0.977	0.971
mAP@50–95	0.796	0.840

Table 1: Test-set detection metrics from the saved model evaluations.

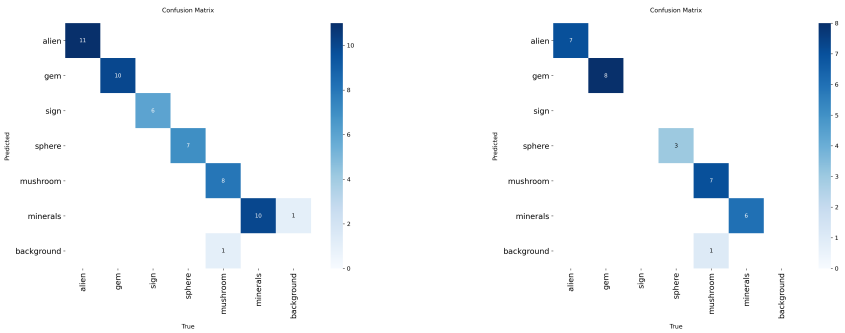


Figure 3: Test-set confusion matrices for the RGB model (left) and the depth model (right). Both models miss one mushroom; the RGB model also produces one false-positive mineral detection. The depth model produces no false positives, and its stop-sign row and column are empty because this class was excluded from the depth labels.

Integrating the detection into ROS proved difficult. Initially, the detection was implemented to run directly inside `image_callback()` in the `CaveExplorer` node. Processing took longer than the 0.2-second image interval, blocking the callback and causing frames to be missed. Inference was therefore moved to a separate node using the `ros_yolo` package. The image callback publishes the selected RGB or depth image to this node, and returned detections trigger `artifact_detections_callback`. This callback retrieves the corresponding image and robot pose at capture time, then coordinates image annotation and artefact localisation, as described in Perception 3. Colour-coded bounding boxes and class labels are published with the images for display in RViz.

Both RGB and depth detections where implemented and the input modality is selected using the `use_depth_detection` parameter in `detection_config.yaml`. Since detection still affected runtime performance, `detection_every_n_images` was added to reduce processing frequency by skipping images between detections. During a cave traversal with a detection confidence threshold of 0.7, no false positives were observed with either model.

Figure 4 illustrates the differences in detection performance between the inputs. Both models detect the nearby mushroom with high confidence, while only RGB detects the stop sign and distant sphere. The stop sign has no distinct depth appearance, and the sphere lies beyond the selected depth range. During exploration, depth detection nevertheless identified all artefacts correctly when they were within range.

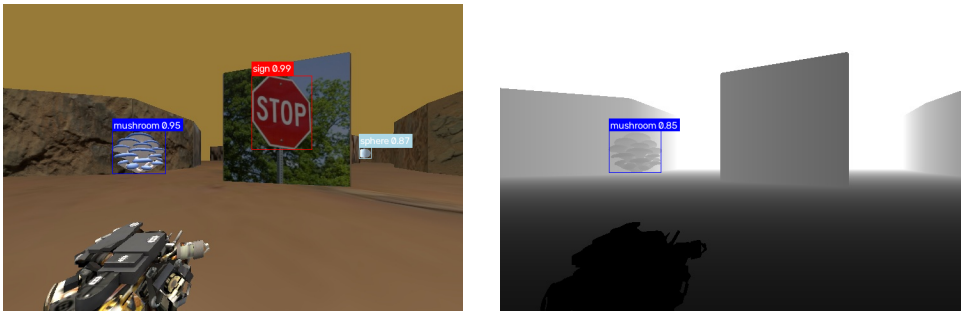


Figure 4: Artefact detections using RGB (left) and depth (right). Both models detect the mushroom; the stop sign and distant sphere are detected only in the RGB image.

4.3 Perception 3: Artefact Localisation and Display

The goal of this task was to estimate the map positions of detected artefacts and display them in RViz. The horizontal centre of each bounding box was used to calculate the artefact's bearing from the camera's field of view and focal length. To calculate the artefact's distance from the camera, first the depth value at the bounding-box centre was retrieved from the depth image. This depth value was then divided by the cosine of this bearing to obtain the distance. Combining this distance and artefact angle with the robot's pose gave the artefact's position in the world frame.

Initially, localisation used the robot's pose at the time the detection result was processed. Since model inference introduced a delay, this produced incorrect positions, particularly while the robot was turning. To correct this, `get_pose_2d()` was extended to accept the image timestamp and retrieve the corresponding pose. TF lookups then sometimes failed with "Lookup would require extrapolation into the future", indicating that the required transform was not yet available. Making the detection callback and pose lookup asynchronous allowed the system to wait for the transform.

```python
def localise_artifact(self, robot_pose, depth_image, detection):

    # Camera parameters
    HORIZONTAL_AOV = 2/3 * math.pi
    IMAGE_WIDTH = 720
    FOCAL_LENGTH = IMAGE_WIDTH / (2 * math.tan(HORIZONTAL_AOV / 2))

    # Calculate artefact angle from detection coordinates
    box = detection.bbox
    cx, cy = round(box.center.position.x),
        round(box.center.position.y)
    bearing = math.atan2((IMAGE_WIDTH/2 - cx), FOCAL_LENGTH)
    absolute_angle = wrap_angle(robot_pose.theta + bearing)

    # Calculate the artifact distance from the depth image
    depth = float(depth_image[cy, cx])
    if not np.isfinite(depth) or depth <= 0:
        return
    distance = depth / math.cos(bearing)

    # Calculate the artifact's position in the world frame.
    point = Point()
    point.x = robot_pose.x + distance * math.cos(absolute_angle)
    point.y = robot_pose.y + distance * math.sin(absolute_angle)
    point.z = 1.0

    # Save the artifact's position to the list
    self.artifact_locations_.append(point)

    # Publish the markers
    self.publish_artifact_markers()
```

Listing 2: Artefact position estimation from a detection and depth image

To archieve usable artifact localizations, repeated observations of the same artefact needed to be combined into a single position estimate. This was achieved by using a Density-Based Spatial Clustering approach (DBSCAN) from the scikit-learn package. This algorithm groups nearby position estimates using a distance threshold and a minimum number of neighbouring points, required to form a core point of a cluster. Several distance thresholds were tested, with 3 metres producing the best grouping in the test scenario. All observations were assigned to the intended artefact clusters in that run, including the two separate chunks of the mineral artefact, which were correctly treated as one artefact. Figure 5 compares the markers before and after clustering.



Figure 5: Artefact localisation markers before clustering (left) and after DBSCAN clustering (right). Repeated observations are combined into a single estimated position for each artefact.

4.4 Planning 1: Autonomously Explore the Cave

The goal for planning task 1 was to autonomously explore the area. Frontier based exploration was used for this, by searching the `map_msg/OccupancyGrid` to find any free cells that were neighbouring unknown cells.

```
def is_frontier_cell(self, pos):
    """Returns if a cell is occupied on the map"""
    if self.occupancy_map[pos] != 0: #FREE space
        return False


    for neighbor in self.get_neighbours(pos):
        if self.occupancy_map[neighbor] == -1: #UNKOWN space
            return True
    return False
```

Listing 3: Frontier cell detection from occupany grid

These were then 'clustered' by checking if there were frontier cells neighbouring each other. These clusters were then returned in a larger array to later calculate the best cluster to approach.

```
def find_frontier_clusters(self):
    """Returns array of frontier clusters"""
    visited = np.zeros(self.occupancy_map.shape, dtype=bool)
    visited_neighbor = np.zeros(self.occupancy_map.shape,
        dtype=bool)
    frontiers = []
    starting_pose = (0,0)
    queue = deque([starting_pose])

    while queue:
        current_cell = queue.popleft()

        if self.is_frontier_cell(current_cell):
            cluster = [] #find all the frontier nodes in a cluster
            inner_queue = deque([current_cell])

            while inner_queue:
                frontier_cell = inner_queue.popleft()
                if visited[(frontier_cell)] == True:
                    continue
                visited[frontier_cell] = True
                cluster.append(frontier_cell)
                for neighbor in self.get_neighbours(frontier_cell):
                    if self.is_frontier_cell(neighbor):
                        inner_queue.append(neighbor)

            if len(cluster) >= 3: # only allow clusters of a certain
                size
                frontiers.append(cluster)

        visited[current_cell] = True
        for neighbor in self.get_neighbours(current_cell):
            if not visited_neighbor[neighbor]:
                visited_neighbor[neighbor] = True
                queue.append(neighbor)

    return frontiers
```

Listing 4: Frontier cell clustering

We used the following code snippit to find the cell indexes of neighbouring cells

```
def get_neighbours(self, pos): # PLANNER 1
    """Returns neighbors to a cell"""
    x,y = pos
    neighbors = []
    for dx in [-1, 0, 1]:
        for dy in [-1, 0, 1]:
            if dx == 0 and dy == 0:
                continue
            nx, ny = x + dx, y + dy
            width, height = self.occupancy_map.shape
            if 0 <= nx < width and 0 <= ny < height:
                neighbors.append((nx,ny))
    return neighbors
```

Listing 5: Find neighbouring cell indexes

Next we had to find the best frontier cluster to approach. We did this by calculating the average position of each cell in a cluster and choosing the nearest option to the robot. However, due to the fact the `map_msg/OccupancyGrid` held data in cells, we must convert these values into the relative world coordinates. This conversion can be seen in lines 16-19 of Listing 6

```python
def planner_nearest_frontier(self):
    """Go to the nearest frontier"""

    pose = self.get_pose_2d()
    frontiers = self.find_frontier_clusters()

    if frontiers: # if there are any unexplored frontiers
        best_frontier = None
        min_dist = 9999999 # A large number

        for clusters in frontiers:
            cluster_world_x = []
            cluster_world_y = []

            for frontier in clusters:
                cluster_world_x.append(self.xlim_[0]+
                                       frontier[0]*self.map_resolution)
                cluster_world_y.append(self.xlim_[0]+
                                       frontier[1]*self.map_resolution)

            centroid_x = np.mean(np.array(cluster_world_x), axis=0)
            centroid_y = np.mean(np.array(cluster_world_y), axis=0)
            centroid = (centroid_x,centroid_y) # average x, y
                coordiantes of cells in a cluster

            distance =
                np.linalg.norm(centroid-np.array((pose.x,pose.y)))

            if distance < min_dist:
                min_dist = distance
                best_frontier = (centroid[0],centroid[1])

        goal = Pose2D(x = best_frontier[0], y = best_frontier[1])
        self.planner_go_to_pose2d(goal)
```

Listing 6: Approach the nearest frontier cluster

4.5 Planning 2: Close-Range Inspection

This task was to approach any artifacts the robot found during its exploration. This was done by checking for artifact detections every main loop, and if found, going to the last seen artifact pose in `self.artifact_locations_` from the perception 3 task.

```
def planning_approach_artifact(self):
    """Go to last seen artifact"""
    robot_pose = self.get_pose_2d()
    artifact_point =
        self.artifact_locations_[len(self.artifact_locations_)-1] #
        Get the last seen artifact
    distance = np.linalg.norm((artifact_point.x,artifact_point.y) -
        (robot_pose.x,robot_pose.y)) - 1 # Near the artifact point
    heading = np.atan(((artifact_point.x - robot_pose.x) /
        (artifact_point.y - robot_pose.y))) # in Rads
    # self.get_logger().info(f'Heading (Degrees):
        {np.degrees(heading)}')
    x_increase = distance * np.cos(heading)
    y_increase = distance * np.sin(heading)
    goal = Pose2D(
        x = robot_pose.x + x_increase,
        y = robot_pose.y + y_increase,
        theta = heading
    )
    self.planner_go_to_pose2d(goal)
    self.visited_artifact_locations_.append(artifact_point)
```

Listing 7: Approach the last seen artifact

4.6 Planning 3: Behaviour Switching

The aim for this task was to avoid visiting the same artifact more than once. Our solution for this was quite simple, whenever the robot visits an artifact, the pose of that artifact is added to an array, as seen at the end of Listing 7, so whenever a new artifact is detected, it checks if the pose of that artifact is already in that array and if so it ignores that detection.

```
1 if self.artifact_found_ and
      self.artifact_locations_[len(self.artifact_locations_)-1] not
      in self.visited_artifact_locations_: # check if a new artifact
      is found
2     self.planner_type_ = PlannerType.APPROACH_ARTIFACT
3 else:
4     self.planner_type_ = PlannerType.NEAREST_FRONTIER
```

Listing 8: Approach the last seen artifact

4.7 Advanced 1: Robust Perception

4.8 Advanced 2: Cave Geometry Analysis

4.9 Advanced 3: Communication Network Deployment

4.10 Advanced 4: Online Roadmap Construction

4.11 Advanced 5: Science Tour Planning

4.12 Advanced 6: Adaptive Scientific Sampling

5 Results

6 Reflection on Teamwork